

Agentic AI

Assignment 1

Author: Prashant Meena (24b0949)

June 22, 2026



Figure 1: Chatbot Image

Output

Figure 2: Outputs

I have tried to follow all good coding practices while coding for example using class, inheritance, error handling, pretty printing, using argparse library instead of hardcoding file name in source code.

My RAG Pipeline didn't hallucinated on any input, though because of using naive chunking and retrieval strategies (which were part of assignment problem statement), it sometimes gave "I don't know" answer to questions in document.

After tweaking some hyperparameters, RAG pipeline was giving way better results than before.

Code Files

code (minimal_rag.py)

```
1 from sentence_transformers import SentenceTransformer
2 from transformers import pipeline
3 from dotenv import load_dotenv
4 from groq import Groq
5 import numpy as np
6 import argparse
7 import os
8
9 class RAG:
10     def __init__(self, file_path: str,
11                  chunk_size: int = 200,
12                  overlap: int = 40,
13                  embedding_model_name: str = 'all-MiniLM-L6-v2'):
14
15         self.file_path = file_path
16         try:
17             with open(self.file_path, "r", encoding="utf-8") as f:
18                 self.text = f.read()
19         except FileNotFoundError:
20             print("Please provide valid supported file")
21             exit(1)
22
23         self.chunk_size = chunk_size
24         self.overlap = overlap
25         if self.overlap >= self.chunk_size:
26             print("overlap must be smaller than chunk_size")
27             exit(1)
28         self.chunks = None
29         self.chunk_text()
30
31         self.embedding_model_name = embedding_model_name
32         self.model = SentenceTransformer(self.
33                                         embedding_model_name)
34         self.chunk_embeddings = None
35         self.embed_text()
36
37         self.generator = pipeline("text2text-generation", model='
38                                     google/flan-t5-base')
39
40     def chunk_text(self):
41         if not self.text:
42             print("No text in chunking stage")
43             exit(1)
44
45         words = self.text.strip().split()
46         self.chunks = []
47
48         start = 0
```

```

47         while start < len(words):
48             end = min(len(words), start + self.chunk_size)
49             self.chunks.append(' '.join(words[start:end]))
50
51             if end == len(words):
52                 break
53
54             start = end - self.overlap
55
56     def embed_text(self):
57         self.chunk_embeddings = self.model.encode(self.chunks,
58             show_progress_bar = True)
59         self.chunk_embeddings = self.chunk_embeddings / np.linalg
60             .norm(self.chunk_embeddings, axis=1, keepdims=True) #
61             (N, d)
62
63     def retrieve(self, query: str, top_k: int = 3) -> list[str]:
64         query_embedding = self.model.encode(query) # (1, d)
65         query_embedding = query_embedding / np.linalg.norm(
66             query_embedding)
67
68         scores = self.chunk_embeddings @ query_embedding.T
69         top_indices = np.argsort(scores)[-top_k:][::-1]
70         return [self.chunks[i] for i in top_indices]
71
72     def rag_prompt(self, query:str) -> str:
73         hypothetical_answer = self.generator(
74             query,
75             max_new_tokens=50
76         )[0]["generated_text"]
77
78         chunks = self.retrieve(hypothetical_answer, top_k = 3)
79         context = "\n\n".join(chunk for chunk in chunks)
80         prompt = f"""
81             Answer the question using ONLY the context below
82             in less than 100 words.
83             If the answer is not in the context, say 'I don't
84             know'.
85
86             context: {context}
87             Question: {query}
88             Answer: """
89         return prompt
90
91 if __name__ == "__main__":
92     parser = argparse.ArgumentParser()
93     parser.add_argument("file_path", type=str)
94     args = parser.parse_args()
95     file_path = args.file_path
96
97     load_dotenv()

```

```

92     client = Groq(api_key= os.getenv("GROQ_API_KEY"))
93
94     rag_object = RAG(file_path)
95
96     while True:
97         print("Enter your query (q to quit): ", end='')
98         query = input().strip()
99
100        if query.lower() == 'q':
101            print("Exiting...")
102            break
103
104        prompt = rag_object.rag_prompt(query)
105
106        chat_completion = client.chat.completions.create(
107            messages=[{
108                "role": "user",
109                "content": prompt
110            }],
111            model="llama-3.3-70b-versatile",
112            max_completion_tokens=400
113        )
114
115        print("Chatbot:", chat_completion.choices[0].message.
116            content)
117        print()

```

Solution 2

Output

Note: I have changed lengths of chunks for comparison. Result obtained here doesn't mean any of the below 3 heuristic is strictly better or worse than other. Sample size of 5 questions and 4000 words having document is a very loose comparison.

```
(groqv) lucky@Prashant:~/groq/week2$ python3 chunking.py leadership.txt
Strategy: fixed_chunk
number of chunks: 101
mean chunk length: 49.76237623762376
min chunk length: 26
max chunk length: 50
-----
Strategy: sentence_chunk
number of chunks: 49
mean chunk length: 82.16326530612245
min chunk length: 8
max chunk length: 130
-----
Strategy: sliding_window_chunk
number of chunks: 80
mean chunk length: 99.7
min chunk length: 76
max chunk length: 100
-----
/home/lucky/groq/groq/lib/python3.10/site-packages/huggingface_hub/file_download.py:949: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use 'force_download=True'.
  warnings.warn(
Batches: 100% | 4/4 [00:01<00:00, 2.02it/s]
Batches: 100% | 2/2 [00:00<00:00, 2.85it/s]
Batches: 100% | 3/3 [00:01<00:00, 2.51it/s]
Strategy 1: Hit Rate = 5/5 = 1.00
Strategy 2: Hit Rate = 4/5 = 0.80
Strategy 3: Hit Rate = 5/5 = 1.00
(groqv) lucky@Prashant:~/groq/week2$
```

Figure 3: Outputs

code (chunking.py)

```
1 from sentence_transformers import SentenceTransformer
2 import numpy as np
3 import argparse
4 import re
5
6 def fixed_chunk(text: str, size: int = 50, overlap: int = 10) ->
   list[str]:
7     if not text:
8         print("No text in chunking stage")
9         exit(1)
10
11     if overlap >= size:
12         print("overlap must be smaller than size")
13         exit(1)
14
15     words = text.strip().split()
16     chunks = []
17
18     start = 0
19     while start < len(words):
20         end = min(len(words), start + size)
21         chunks.append(' '.join(words[start:end]))
22
```

```

23         if end == len(words):
24             break
25
26         start = end - overlap
27
28     return chunks
29
30 def sentence_chunk(text: str) -> list[str]:
31     if not text:
32         print("No text in chunking stage")
33         exit(1)
34
35     raw_sentences = [s.strip() for s in re.split(r'[.?!]', text.
36                 strip()) if s.strip()]
37     chunks = []
38     for start in range(0, len(raw_sentences), 5):
39         end = min(len(raw_sentences), start + 5) # [start, end)
40         chunk = " ".join(raw_sentences[start:end])
41         chunks.append(chunk)
42     return chunks
43
44 def sliding_window_chunk(text: str, window: int = 100, step: int
45     = 50) -> list[str]:
46     if not text:
47         print("No text in chunking stage")
48         exit(1)
49
50     raw_words = text.strip().split()
51     chunks = []
52
53     start = 0
54     while start < len(raw_words):
55         end = min(len(raw_words), start + window)
56         chunk = " ".join(raw_words[start:end])
57         chunks.append(chunk)
58
59         if end == len(raw_words):
60             break
61
62         start += step
63
64     return chunks
65
66 def A(text: str):
67     chunk1 = fixed_chunk(text)
68     N = len(chunk1)
69     assert N != 0
70     mean = sum(len(chunk.split()) for chunk in chunk1) / N
71     mini = min(len(chunk.split()) for chunk in chunk1)
72     maxi = max(len(chunk.split()) for chunk in chunk1)
73     print(f"Strategy: fixed_chunk")

```

```

72     print(f"number of chunks: {N}")
73     print(f"mean chunk length: {mean}")
74     print(f"min chunk length: {mini}")
75     print(f"max chunk length: {maxi}")
76     print("-" * 50)
77
78     return chunk1
79
80 def B(text: str):
81     chunk2 = sentence_chunk(text)
82     N = len(chunk2)
83     assert N != 0
84     mean = sum(len(chunk.split()) for chunk in chunk2) / N
85     mini = min(len(chunk.split()) for chunk in chunk2)
86     maxi = max(len(chunk.split()) for chunk in chunk2)
87     print(f"Strategy: sentence_chunk")
88     print(f"number of chunks: {N}")
89     print(f"mean chunk length: {mean}")
90     print(f"min chunk length: {mini}")
91     print(f"max chunk length: {maxi}")
92     print("-" * 50)
93
94     return chunk2
95
96 def C(text: str):
97     chunk3 = sliding_window_chunk(text)
98     N = len(chunk3)
99     assert N != 0
100    mean = sum(len(chunk.split()) for chunk in chunk3) / N
101    mini = min(len(chunk.split()) for chunk in chunk3)
102    maxi = max(len(chunk.split()) for chunk in chunk3)
103    print(f"Strategy: sliding_window_chunk")
104    print(f"number of chunks: {N}")
105    print(f"mean chunk length: {mean}")
106    print(f"min chunk length: {mini}")
107    print(f"max chunk length: {maxi}")
108    print("-" * 50)
109
110    return chunk3
111
112 if __name__ == "__main__":
113     parser = argparse.ArgumentParser()
114     parser.add_argument("file_path", type=str)
115     args = parser.parse_args()
116     file_path = args.file_path
117
118     try:
119         with open(file_path, "r", encoding="utf-8") as f:
120             text = f.read()
121     except FileNotFoundError:
122         print("Please provide valid supported file")

```

```

123         exit(1)
124
125     chunk1 = A(text)
126     chunk2 = B(text)
127     chunk3 = C(text)
128
129     embedder = SentenceTransformer('all-MiniLM-L6-v2')
130
131     chunk1_embeddings = embedder.encode(chunk1, show_progress_bar
        = True)
132     chunk1_embeddings = chunk1_embeddings / np.linalg.norm(
        chunk1_embeddings, axis=1, keepdims=True)
133
134     chunk2_embeddings = embedder.encode(chunk2, show_progress_bar
        = True)
135     chunk2_embeddings = chunk2_embeddings / np.linalg.norm(
        chunk2_embeddings, axis=1, keepdims=True)
136
137     chunk3_embeddings = embedder.encode(chunk3, show_progress_bar
        = True)
138     chunk3_embeddings = chunk3_embeddings / np.linalg.norm(
        chunk3_embeddings, axis=1, keepdims=True)
139
140     def retrieve_top_indices(query: str, chunk_embeddings, top_k:
        int = 1) -> list[str]:
141         global embedder
142
143         query_embedding = embedder.encode(query)
144         query_embedding = query_embedding / np.linalg.norm(
            query_embedding)
145
146         scores = chunk_embeddings @ query_embedding.T
147         top_indices = np.argsort(scores)[-top_k:][::-1]
148         return top_indices
149
150     questionnaire = {
151         1: {
152             "question": "What are the five elements of emotional
                intelligence according to Goleman?",
153             "answer": [
154                 "self-awareness",
155                 "self-regulation",
156                 "motivation",
157                 "empathy",
158                 "social skills"
159             ]
160         },
161
162         2: {
163             "question": "Which leadership styles are associated
                with Kurt Lewin's behavioural theory?",

```

```

164         "answer": [
165             "autocratic",
166             "democratic",
167             "laissez-faire"
168         ]
169     },
170
171     3: {
172         "question": "Which company in France studied
173             emotional intelligence in its workforce?",
174         "answer": [
175             "sanofi"
176         ]
177     },
178
179     4: {
180         "question": "Which company uses peer reviews for
181             manager feedback?",
182         "answer": [
183             "google"
184         ]
185     },
186
187     5: {
188         "question": "Which car company is mentioned as
189             investing in leadership training?",
190         "answer": [
191             "bentley motors"
192         ]
193     }
194 }
195 total = len(questionnaire)
196
197 for i, chunk_embeddings in enumerate([chunk1_embeddings,
198     chunk2_embeddings, chunk3_embeddings]):
199     Hits = 0
200     for item in questionnaire.values():
201         query = item["question"]
202         answers = item["answer"]
203
204         top_indices = retrieve_top_indices(query,
205             chunk_embeddings)
206
207         if i == 0:
208             top_k_chunks = [chunk1[idx] for idx in
209                 top_indices]
210         elif i == 1:
211             top_k_chunks = [chunk2[idx] for idx in
212                 top_indices]
213         else:

```

```

207         top_k_chunks = [chunk3[idx] for idx in
208                             top_indices]
209     context = " ".join(top_k_chunks).lower()
210     # print(f"Strategy {i+1}: context: {context}") # for
211     # debugging
212     if all(ans.lower() in context for ans in answers):
213         Hits += 1
214
215     print(f"Strategy {i+1}: Hit Rate = {Hits}/{total} = {Hits
216           /total:.2f}")

```

Solution 3

code (decision.py)

```
1 def recommend_approach(scenario: dict) -> str:
2     data_changes = scenario["data_changes_frequently"]
3     specific_format = scenario["need_specific_output_format"]
4     budget = scenario["budget"]
5     latency = scenario["latency_sensitive"]
6     knowledge = scenario["knowledge_type"]
7
8     if knowledge == "both":
9         if budget in ["medium", "high"]:
10             return "RAG + Fine-Tuning: You need dynamic facts
11                 retrieval plus customized behavior/output style."
12         else:
13             return "RAG: Both would help, but budget is low, so
14                 prioritize retrieval over training."
15
16     elif knowledge == "factual":
17         if data_changes:
18             return "RAG: Knowledge changes frequently, so
19                 external retrieval is better than retraining."
20         elif latency and budget == "low":
21             return "Prompt Engineering only: Facts are stable and
22                 constraints are simple, so prompting may be
23                 enough."
24         else:
25             return "RAG: External knowledge retrieval improves
26                 factual accuracy."
27
28     elif knowledge == "behavioral":
29         if specific_format:
30             if budget == "high":
31                 return "Fine-Tuning: Strong formatting/style
32                     consistency benefits from model training."
33             else:
34                 return "Prompt Engineering only: Formatting can
35                     be controlled through carefully designed
36                     prompts."
37         else:
38             return "Prompt Engineering only: Behavior tweaks
39                 alone often don't require fine-tuning."
40
41     return "Prompt Engineering only: Default lightweight solution
42         ."
43
44 if __name__ == "__main__":
45     scenarios = [
46         {
```

```

37         "name": "Customer support chatbot with daily policy
38             updates",
39         "data_changes_frequently": True,
40         "need_specific_output_format": False,
41         "budget": "medium",
42         "latency_sensitive": False,
43         "knowledge_type": "factual"
44     },
45     {
46         "name": "Legal document formatter",
47         "data_changes_frequently": False,
48         "need_specific_output_format": True,
49         "budget": "high",
50         "latency_sensitive": True,
51         "knowledge_type": "behavioral"
52     },
53     {
54         "name": "Personal tutor with company-specific
55             documents and teaching style",
56         "data_changes_frequently": True,
57         "need_specific_output_format": True,
58         "budget": "high",
59         "latency_sensitive": False,
60         "knowledge_type": "both"
61     },
62     {
63         "name": "Simple email rewriter",
64         "data_changes_frequently": False,
65         "need_specific_output_format": True,
66         "budget": "low",
67         "latency_sensitive": True,
68         "knowledge_type": "behavioral"
69     },
70     {
71         "name": "Static FAQ bot",
72         "data_changes_frequently": False,
73         "need_specific_output_format": False,
74         "budget": "low",
75         "latency_sensitive": True,
76         "knowledge_type": "factual"
77     }
78 ]
79
80 ]
81
82 for s in scenarios:
83     print(f"{s['name']} -> {recommend_approach(s)}")
84     print(" ")

```

code (test.py)

```
(groqv) lucky@Prashant:~/groq/week2$ python3 test.py leadership.txt
/home/lucky/groq/groqv/lib/python3.10/site-packages/huggingface_hub/file_download.py:949: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use 'force_download=True'.
  warnings.warn(
Batches: 100% | 1/1 [00:02<00:00, 2.13s/it]
answerable: green | LLM: Management training and development is considered vital as it helps companies align their management development and strategic goals, leading to success in competitive business environments and enabling effective organisational management for a positive workplace.

answerable: green | LLM: Effective managers need practical, interpersonal, and conceptual skills.

answerable: green | LLM: Yes, according to the essay, leadership can be trained. It is stated that "leadership can be trained" and "leadership capabilities can be nurtured through training and experience."

answerable: red | LLM: I don't know.

answerable: red | LLM: I don't know.

answerable: red | LLM: I don't know.

Saved plot as rag_scores.png
(groqv) lucky@Prashant:~/groq/week2$
```

Figure 4: outputs

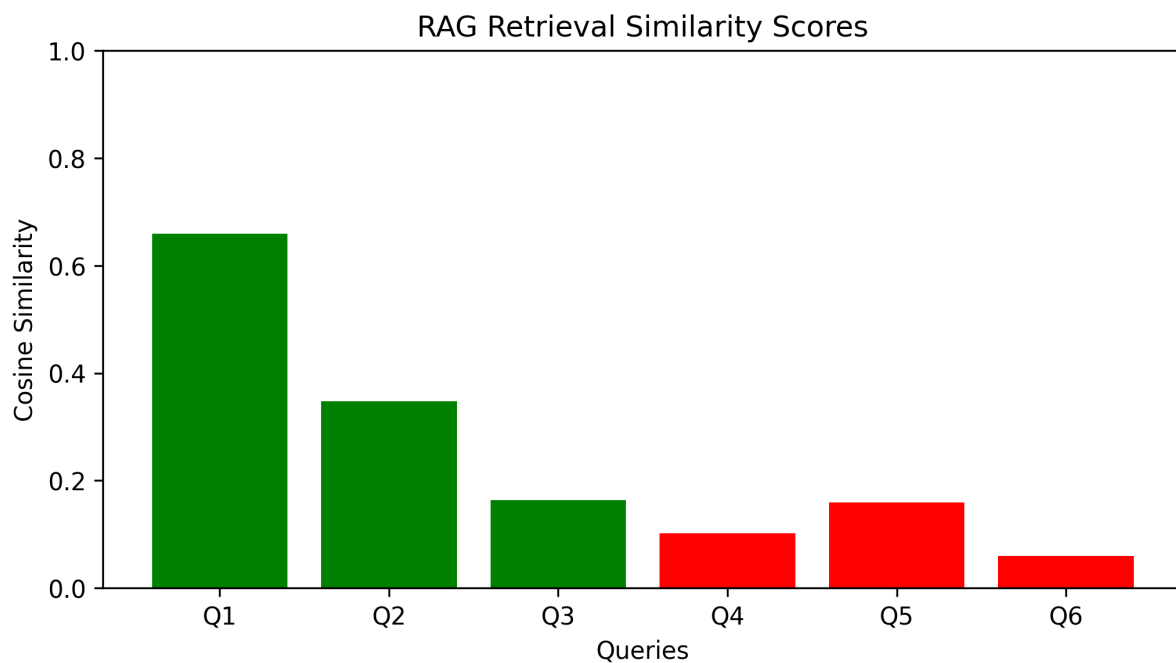


Figure 5: rag_scores

```
1 import matplotlib.pyplot as plt
2 from dotenv import load_dotenv
3 from minimal_rag import *
4 from groq import Groq
5 import numpy as np
6 import argparse
7 import os
8
9 class TWEAKED_RAG(RAG):
10     def retrieve(self, query: str, top_k: int = 3) -> tuple[list
11         [str], float]:
12         query_embedding = self.model.encode(query)
```

```

12         query_embedding = query_embedding / np.linalg.norm(
13             query_embedding)
14
15         scores = self.chunk_embeddings @ query_embedding.T
16         top_indices = np.argsort(scores)[-top_k:][::-1]
17         return [self.chunks[i] for i in top_indices], scores[
18             top_indices[0]]
19
20     def rag_prompt(self, query:str):
21         hypothetical_answer = self.generator(
22             query,
23             max_new_tokens=50
24         )[0]["generated_text"]
25
26         chunks, max_similarity = self.retrieve(
27             hypothetical_answer, top_k = 3)
28         context = "\n\n".join(chunk for chunk in chunks)
29         prompt = f"""
30             Answer the question using ONLY the context below
31             in less than 100 words.
32             If the answer is not in the context, say 'I don't
33             know'.
34
35             context: {context}
36             Question: {query}
37             Answer: ""
38
39         top_chunk = chunks[0] if chunks else ""
40
41         return prompt, top_chunk, max_similarity
42
43 if __name__ == "__main__":
44     parser = argparse.ArgumentParser()
45     parser.add_argument("file_path", type=str)
46     args = parser.parse_args()
47     file_path = args.file_path
48
49     load_dotenv()
50     client = Groq(api_key= os.getenv("GROQ_API_KEY"))
51
52     rag_object = TWEAKED_RAG(file_path)
53
54     queries = ["Q1", "Q2", "Q3", "Q4", "Q5", "Q6"]
55     questionnaire = [
56         "Why is management training considered important in
57         organisations?",
58         "What three types of skills do effective managers need
59         according to the essay?",
60         "Can managers be trained to become leaders according to
61         the essay?",
62         "What is the time complexity of quicksort in the average

```

```

55         case?",
56         "Who invented the Python programming language?",
57         "How does blockchain prevent double spending?"
58     ]
59     colors = [
60         "green",
61         "green",
62         "green",
63         "red",
64         "red",
65         "red"
66     ]
67     LLM_RESPONSES = []
68     TOP_CHUNK = []
69     SIMILARITY = []
70
71     for i, query in enumerate(questionnaire):
72         prompt, top_chunk, max_similarity = rag_object.rag_prompt(
73             query)
74
75         chat_completion = client.chat.completions.create(
76             messages=[{
77                 "role": "user",
78                 "content": prompt
79             }],
80             model="llama-3.3-70b-versatile",
81             max_completion_tokens=400
82         )
83
84         LLM_RESPONSES.append(chat_completion.choices[0].message.
85             content)
86         print(f"answerable: {colors[i]} | LLM: {LLM_RESPONSES
87             [-1]}", end="\n\n")
88         TOP_CHUNK.append(top_chunk)
89         SIMILARITY.append(max_similarity)
90
91     plt.figure(figsize=(8,4))
92     plt.bar(queries, SIMILARITY, color=colors)
93     plt.xlabel("Queries")
94     plt.ylabel("Cosine Similarity")
95     plt.title("RAG Retrieval Similarity Scores")
96     plt.ylim(0,1)
97     plt.savefig("rag_scores.png", dpi=300, bbox_inches="tight")
98     print("Saved plot as rag_scores.png")

```